SECURE CODING REVIEW REPORT

Project Title

Secure Coding Review of a Python Login System

Name

Aditya Sai Teja Ravilisetti

Internship

CodeAlpha Cyber Security Internship

Task

Task 3: Secure Coding Review

Objective

The objective of this project is to perform a security review of a Python-based login application, identify security vulnerabilities, analyze their impact, and recommend secure coding practices to improve the application's security.

Tools Used

- Python 3
- Visual Studio Code (VS Code)
- SQLite Database
- Bandit Static Security Analyzer

Application Overview

A simple login application was developed using Python and SQLite. The application accepts a username and password from the user and verifies the credentials against records stored in a database.

The application was intentionally reviewed for security weaknesses to understand common vulnerabilities and their remediation techniques.

Security Analysis

Vulnerability 1: SQL Injection

Description

The application constructs SQL queries using direct string formatting with user-supplied input. This allows attackers to manipulate the query and potentially bypass authentication.

Vulnerable Code

query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"

Risk

- Authentication bypass
- Unauthorized access
- Database manipulation

Severity

High

Detection Method

Detected using Bandit Static Security Analyzer and manual code review.

Vulnerability 2: Plain Text Password Storage

Description

User passwords are stored in the database without encryption or hashing.

Risk

- Password exposure
- Credential theft
- Data breach

Severity

High

Vulnerability 3: Missing Error Handling

Description

The application does not properly handle database-related exceptions.

Risk

- Application crashes
- Information disclosure
- Poor user experience

Severity

Medium

Vulnerability 4: Lack of Input Validation

Description

The application accepts user input without validation.

Risk

- Invalid data processing
- Increased attack surface

Severity

Medium

Bandit Analysis Result

The Bandit security scanner detected a potential SQL Injection vulnerability in the login application.

Issue Identified:

- B608: Possible SQL Injection vector through string-based query construction.

The scan successfully highlighted insecure SQL query creation practices and confirmed the presence of a security weakness.

Remediation and Secure Coding Practices

The following improvements were implemented:

1. Replaced string-based SQL queries with parameterized queries.
2. Added exception handling using try-except blocks.
3. Recommended password hashing using secure algorithms such as bcrypt.
4. Suggested input validation to restrict malicious or invalid data.

Secure Query Example

```
query = "SELECT * FROM users WHERE username=? AND password=?"
cursor.execute(query, (username, password))
```

This approach prevents SQL Injection attacks by separating user input from SQL commands.

Conclusion

The secure coding review successfully identified multiple security vulnerabilities within the Python login application. Static analysis using Bandit and manual inspection were used to detect security issues. Appropriate remediation techniques were applied to improve application security. This project demonstrates the importance of secure coding practices in preventing common cyber security vulnerabilities such as SQL Injection and insecure password handling.